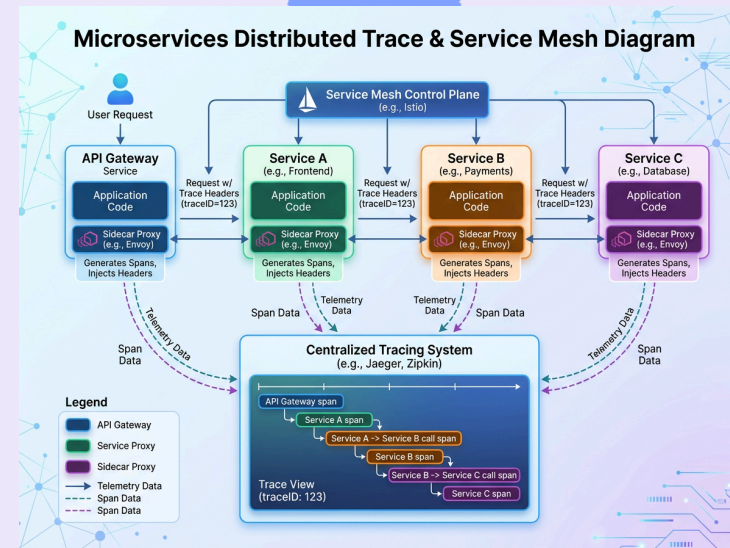


Infraestrutura Ágil, SOA e Microsserviços

Evolução e desafios na construção de sistemas distribuídos modernos.



INSTITUIÇÃO EXECUTORA



COORDENADORA



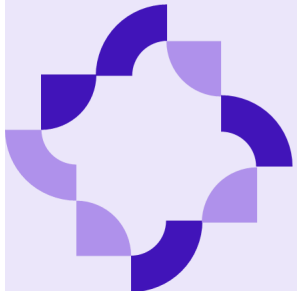
APOIO





Sumário

- Conceito de Infraestrutura Ágil
- SOA e Arquitetura de Microserviços
- Ferramentas: IaC, Containers e CI/CD
- Desafios em Redes, Segurança e Observabilidade
- Decisão: SOA versus Microserviços



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Infraestrutura Ágil

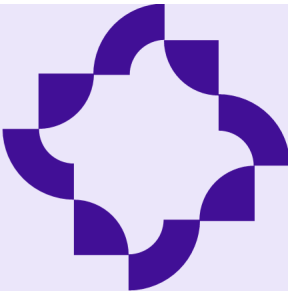
A **infraestrutura ágil** não se limita à velocidade de provisionamento, mas à capacidade de responder a mudanças através de uma arquitetura flexível e automatizada, rompendo com silos tradicionais (Dev vs Ops e hardware estático).

Pilares Fundamentais:

- **Automação:** Eliminação de processos manuais repetitivos.
- **Elasticidade:** Capacidade de escalar recursos horizontal e verticalmente de forma dinâmica.
- **Padronização:** Uso de ambientes idênticos para desenvolvimento, teste e produção.
- **Cultura DevOps:** Integração contínua entre desenvolvimento e operação de sistemas.



Automação de armazém inteligente: robôs, drones e dados para agilidade.

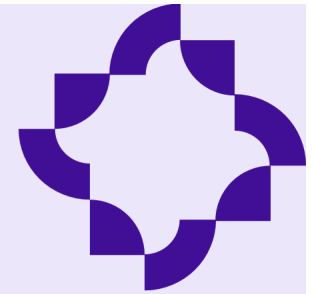


Do Monolito aos Serviços

A evolução tecnológica exigiu a transição de aplicações monolíticas para arquiteturas distribuídas para suportar a escala e a agilidade modernas.

Característica	Arquitetura Monolítica	Arquitetura Orientada a Serviços
Deploy	Unitário e pesado	Independente por serviço
Escalabilidade	Toda a aplicação deve escalar	Apenas o serviço demandado escala
Falhas	Uma falha pode derrubar todo o sistema	Falhas são isoladas em serviços específicos
Tecnologia	Stack tecnológica única	Stack poliglota





Arquitetura Orientada a Serviços (SOA)

SOA é um estilo arquitetural que foca na integração de aplicações através de serviços reutilizáveis com interfaces bem definidas. É comumente associada à integração de sistemas legado em grandes corporações. Seus princípios incluem interoperabilidade, reutilização, acoplamento fraco e contratos de serviço.





O Enterprise Service Bus (ESB)

- **Mediador Central:** O ESB (Barramento de Serviços Corporativos) é o componente central em muitas implementações SOA, atuando como um mediador de comunicações.
- **Roteamento Eficaz:** Direciona mensagens de forma inteligente entre provedores e consumidores de serviços, garantindo a entrega correta.
- **Transformação de Dados:** Realiza a conversão de formatos de dados, como de XML para JSON, assegurando a compatibilidade entre sistemas.
- **Tradução de Protocolos:** Permite a comunicação entre sistemas que utilizam diferentes protocolos, traduzindo, por exemplo, HTTP para JMS.

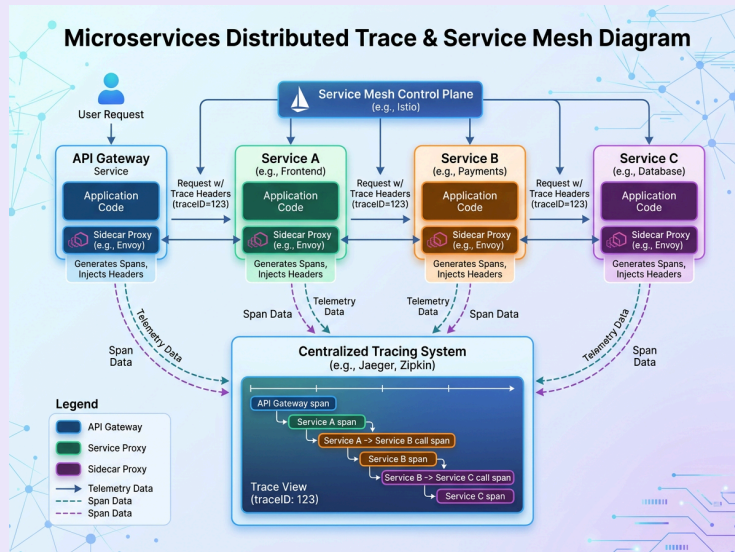


Diagrama de arquitetura de microsserviços com service mesh e rastreamento.

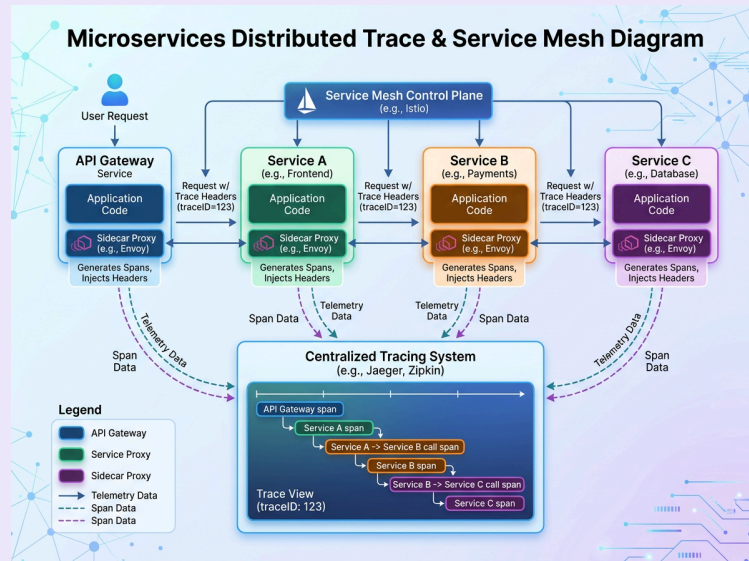
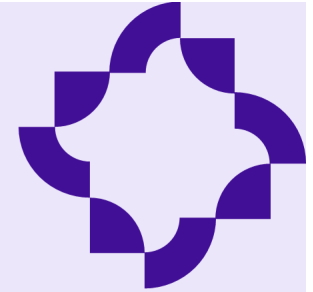
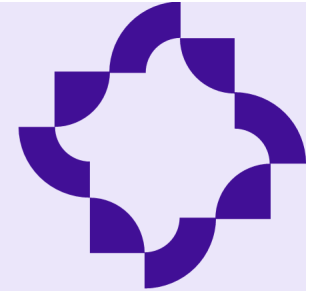


Diagrama de arquitetura de microsserviços com service mesh e rastreamento.

O Enterprise Service Bus (ESB)

- **Segurança e Monitoramento:** Centraliza a autenticação e o logging de mensagens, aprimorando a segurança e a rastreabilidade da comunicação.



Arquitetura de Microsserviços

- **Descentralização:** Ao contrário do SOA, não há um barramento central de serviços; a lógica de negócios reside nos próprios *endpoints*.
- **Bancos de Dados Próprios:** Cada microsserviço gerencia seu *próprio banco de dados*, evitando acoplamento entre os serviços.
- **Comunicação Leve:** Utilizam frequentemente APIs RESTful síncronas ou mensageria assíncrona para interações flexíveis.
- **Independência de Ciclo de Vida:** Cada serviço pode ser desenvolvido, implantado e escalado de forma independente.

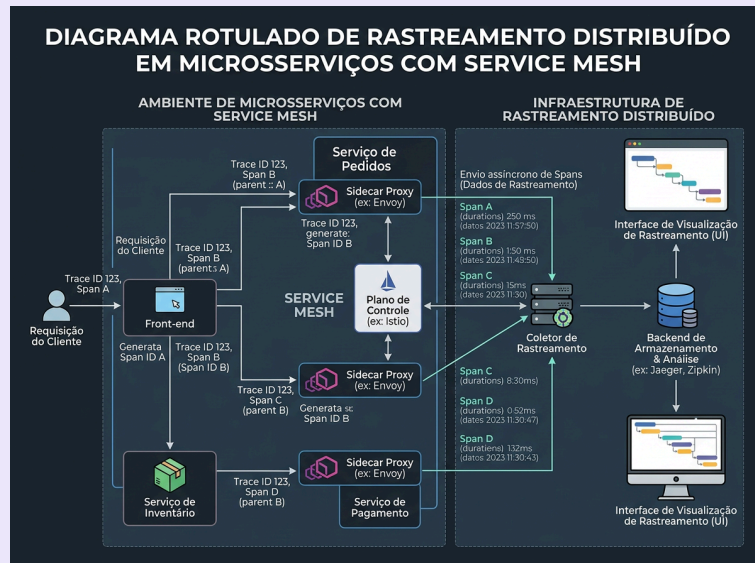
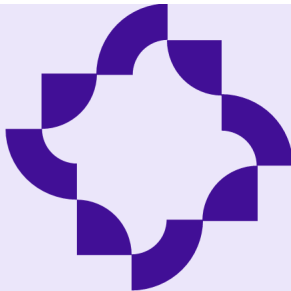


Diagrama de rastreamento distribuído em microsserviços com service mesh.

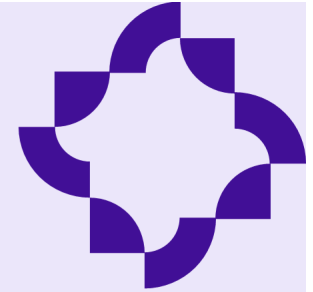


SOA vs. Microserviços

Embora ambos utilizem serviços, a filosofia de implementação e o escopo são distintos.

Atributo	SOA	Microserviços
Escopo	Corporativo	Aplicação
Comunicação	Inteligente (ESB com lógica)	Simples (HTTP/gRPC/Message Queues)
Dados	Compartilhados entre serviços	Privados por serviço
Governança	Centralizada e rígida	Descentralizada e flexível
Tamanho	Serviços maiores	Serviços mínimos





Infraestrutura como Código (IaC)

- A **IaC** gerencia a infraestrutura de TI através de arquivos de definição legíveis por máquina, automatizando o provisionamento.
- **Reprodutibilidade**: Ambientes idênticos podem ser recriados de forma consistente em minutos, garantindo uniformidade.
- **Versionamento**: Arquivos de infraestrutura são armazenados em sistemas como Git, permitindo auditoria completa e *rollbacks* eficientes.
- **Idempotência**: A aplicação da configuração sempre resulta no mesmo estado final desejado, independentemente do estado inicial do sistema.

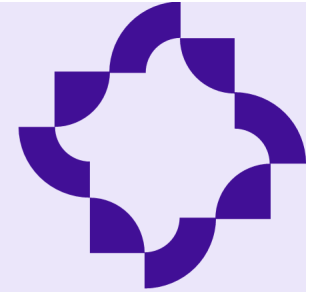
Redes digitais interconectadas simbolizam a infraestrutura de TI automatizada.



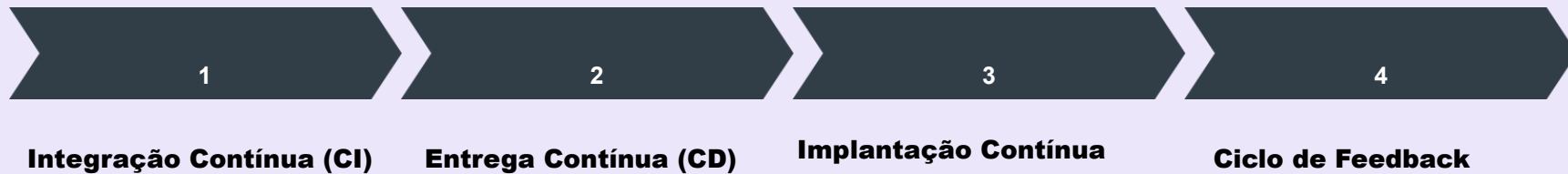
Redes digitais interconectadas simbolizam a infraestrutura de TI automatizada.

Infraestrutura como Código (IaC)

- **Ferramentas comuns:** Incluem Terraform, Ansible, CloudFormation e Pulumi, que facilitam a orquestração e automação da infraestrutura.



O Ciclo de CI/CD



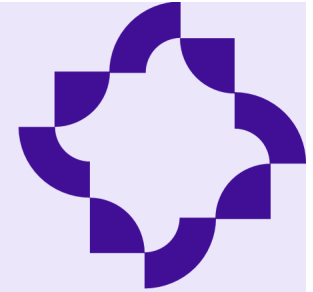
Desenvolvedores integram o código-fonte em um repositório compartilhado **frequentemente**, idealmente várias vezes ao dia. Cada *commit* aciona testes automatizados para validar a integridade do código e detectar conflitos precocemente.

Após a validação bem-sucedida da CI, o código é **automaticamente preparado** para ser implantado. Isso envolve a construção de artefatos, execução de testes adicionais e empacotamento, tornando-o pronto para *deploy* em ambientes como *staging* ou produção.

O *deploy* em produção ocorre de forma **totalmente automática** após o código passar por todos os *portões de qualidade* e testes nos ambientes anteriores. Elimina a intervenção manual, acelerando o tempo de lançamento.

O monitoramento contínuo dos serviços em produção é crucial. Dados de performance, erros e comportamento do usuário alimentam o ciclo de desenvolvimento **em tempo real**, permitindo ajustes rápidos e melhorias contínuas.





Containers e Orquestração

Isolamento e Eficiência

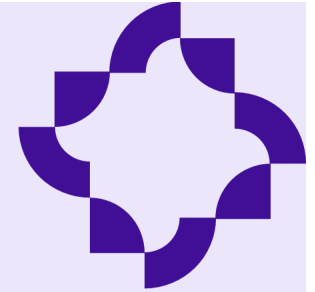
Containers fornecem o isolamento necessário para que os microsserviços rodem de forma consistente em qualquer ambiente. Cada serviço executa em seu próprio *namespace*, com todas as suas dependências empacotadas. São mais leves que as VMs, compartilhando o *kernel* do sistema operacional e oferecendo maior eficiência no uso de recursos.

Kubernetes (K8s) Orquestração

O **Kubernetes (K8s)** atua como o orquestrador padrão, gerenciando o ciclo de vida dos containers. Suas funcionalidades incluem escalonamento automático de serviços, *service discovery* (descoberta de serviços) e *self-healing* (auto-recuperação) de aplicações, garantindo alta disponibilidade e resiliência.

Princípio da Imutabilidade

Uma característica fundamental é a **imutabilidade**: uma vez que uma imagem de container é gerada, ela não deve ser alterada durante sua execução. Quaisquer modificações no código ou nas configurações exigem a criação e implantação de uma nova imagem de container, promovendo consistência e rastreabilidade.



Desafios de Redes e Segurança



Latência e Discovery

A comunicação entre serviços ocorre via rede. Protocolos como **gRPC** são empregados para diminuir o *overhead* de comunicação. Para IPs dinâmicos, são utilizados *service discovery* com ferramentas como Consul ou DNS do Kubernetes (K8s DNS).



Segurança Zero Trust

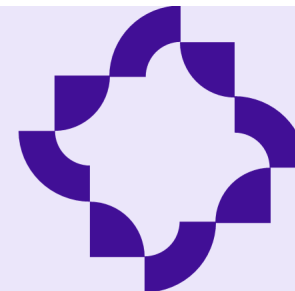
A premissa da rede interna não ser segura exige que cada requisição seja **autenticada** e **autorizada** obrigatoriamente. Métodos como *mTLS* (mutual TLS) e JWT (*JSON Web Tokens*) são cruciais para esta abordagem de segurança.



Resiliência Operacional

Para evitar falhas em cascata em um ambiente distribuído, aplica-se o padrão **Circuit Breaker**. Ele interrompe temporariamente as chamadas a um serviço que está falhando, protegendo o sistema de colapsos generalizados.





Observabilidade: Os Três Pilares

Logging

Registros detalhados de eventos são cruciais em sistemas distribuídos. Ferramentas como **ELK Stack** (Elasticsearch, Logstash, Kibana) são utilizadas para agregação, busca e análise desses logs, permitindo entender o que aconteceu em cada ponto da infraestrutura.

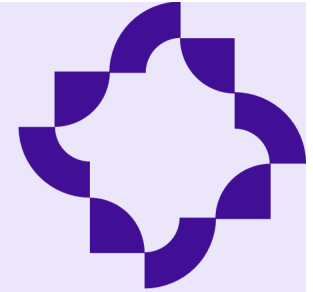
Métricas

Dados numéricos agregados ao longo do tempo fornecem *insights* sobre o desempenho e a saúde dos serviços. **Prometheus** para coleta e **Grafana** para visualização são comumente usados para monitorar CPU, RAM, Latência e Taxa de Erro.

Tracing



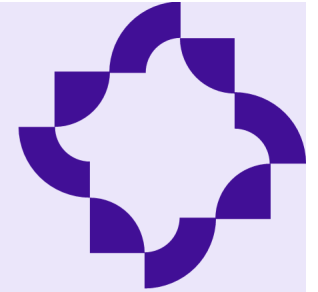
O rastreamento de requisições através de múltiplos serviços permite visualizar o fluxo completo de uma transação. Ferramentas como **Jaeger** ou **Zipkin** são essenciais para identificar gargalos de desempenho e falhas em chamadas entre microsserviços.



Relacione as colunas

- | | | | |
|---|---------------------------|-----|---|
| 1 | ESB | () | Característica fundamental de Microserviços para evitar acoplamento. |
| 2 | Idempotência | () | Componente central de integração em arquiteturas SOA. |
| 3 | Observabilidade | () | Conjunto de práticas incluindo Logs, Métricas e Tracing. |
| 4 | Descentralização de Dados | () | Princípio de IaC que garante o mesmo estado final após múltiplas execuções. |

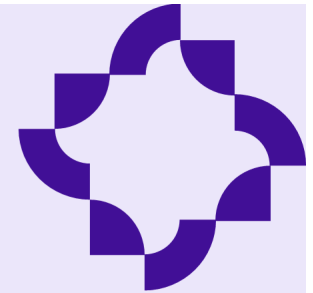




Relacione as colunas

- | | | | |
|---|---------------------------|---|---|
| 1 | ESB | 4 | Característica fundamental de Microserviços para evitar acoplamento. |
| 2 | Idempotência | 1 | Componente central de integração em arquiteturas SOA. |
| 3 | Observabilidade | 3 | Conjunto de práticas incluindo Logs, Métricas e Tracing. |
| 4 | Descentralização de Dados | 2 | Princípio de IaC que garante o mesmo estado final após múltiplas execuções. |





Discuta com o colega ao lado

Considerando os cenários de uma startup de crescimento rápido e um grande banco com sistemas legados: qual arquitetura (SOA ou Microserviços) você recomendaria para cada um?

Justifique suas escolhas com base em critérios técnicos como escalabilidade, complexidade de integração, velocidade de desenvolvimento e custo de manutenção.





Conclusão

- 
- Infraestrutura Ágil rompe silos Dev/Ops, priorizando automação, elasticidade, padronização e cultura DevOps para resposta ágil.
 - SOA integra sistemas legados com ESB, usando serviços reutilizáveis, interoperabilidade via SOAP/REST e acoplamento fraco.
 - Microserviços descentralizam, com serviços autônomos, bancos de dados próprios e comunicação leve (RESTful, mensageria assíncrona).
 - IaC (Terraform, Ansible), Containers (Docker) e Orquestração (Kubernetes) são cruciais para pipelines CI/CD e resiliência.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO